

**Министерство науки и высшего образования Российской
Федерации ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО**

**Лабораторная работа №3
по дисциплине
“Фронт-энд разработка”**

Семестр IV

**Выполнил:
студент
Ковров Евгений Вадимович
гр. J3212
ИСУ 466206**

**Санкт-Петербург
2026**

1 Цель работы

Освоить разработку одностраничных веб-приложений (SPA) с использованием фреймворка Vue.js, а также закрепить навыки работы с маршрутизацией, внешним API, компонентным подходом и переиспользуемой логикой (composables) на примере миграции ранее разработанного приложения.

2 Ход работы

В рамках лабораторной работы было выполнено поэтапное преобразование существующего приложения «Контрамарка», реализованного в предыдущих работах (ЛР1–ЛР2), в одностраничное приложение (SPA) с использованием Vue 3.

2.1 Инициализация SPA-приложения

Был создан новый проект на Vue 3 с использованием сборщика Vite. Подключены необходимые зависимости:

- Vue Router — для маршрутизации;
- Axios — для работы с внешним API (json-server).

Сформирована базовая структура проекта:

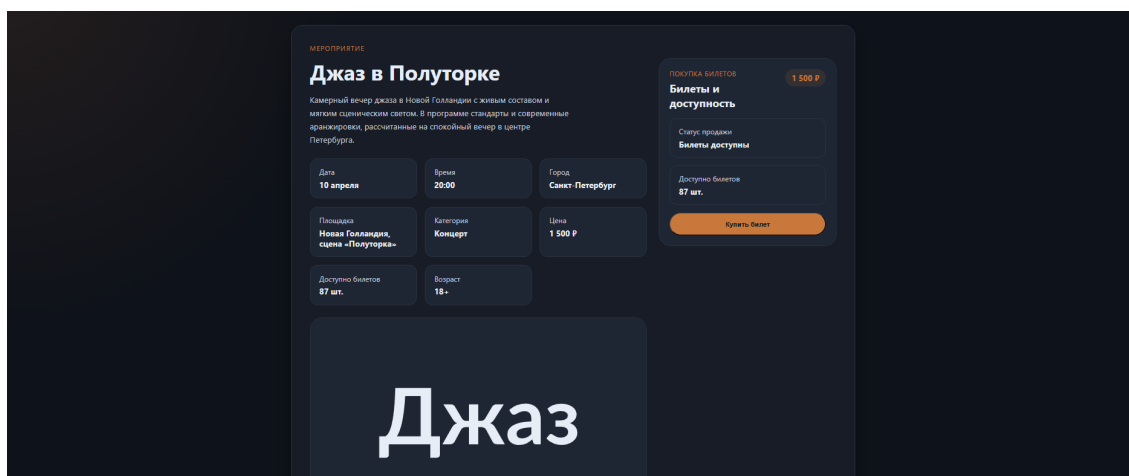
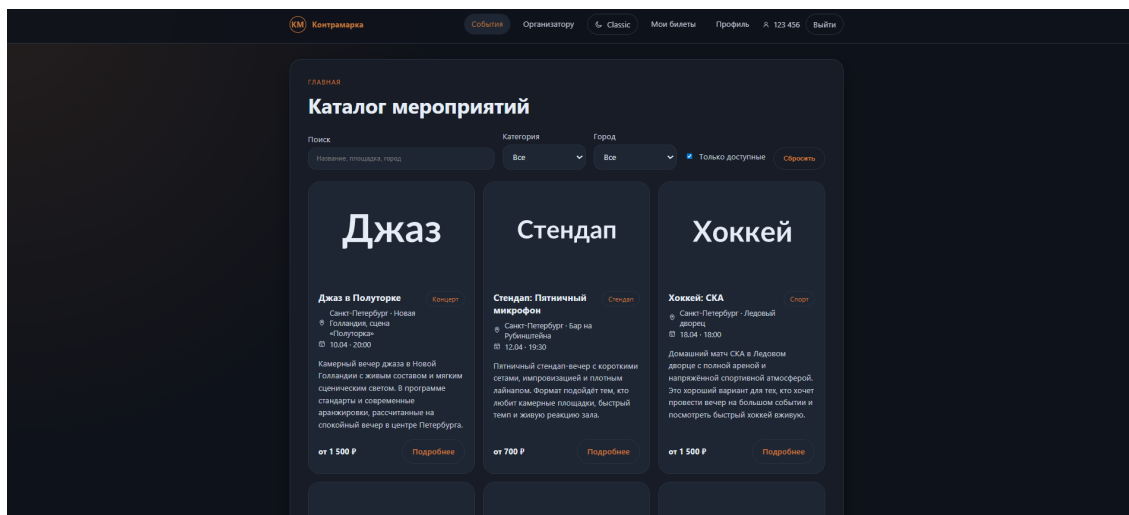
- `views` — страницы приложения;
- `components` — переиспользуемые UI-компоненты;
- `api` — слой работы с HTTP-запросами;
- `composables` — переиспользуемая бизнес-логика;
- `router` — конфигурация маршрутов.

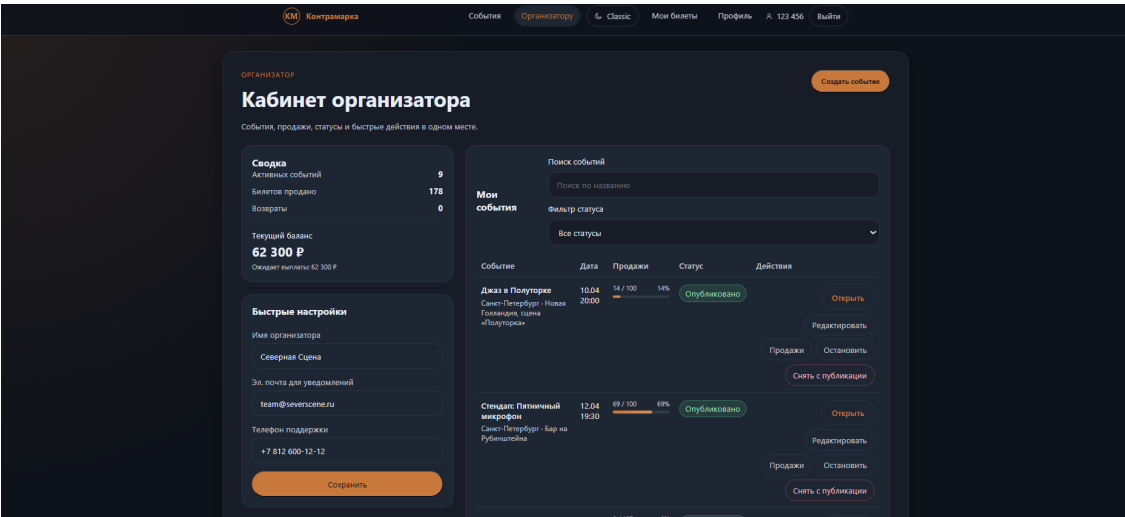
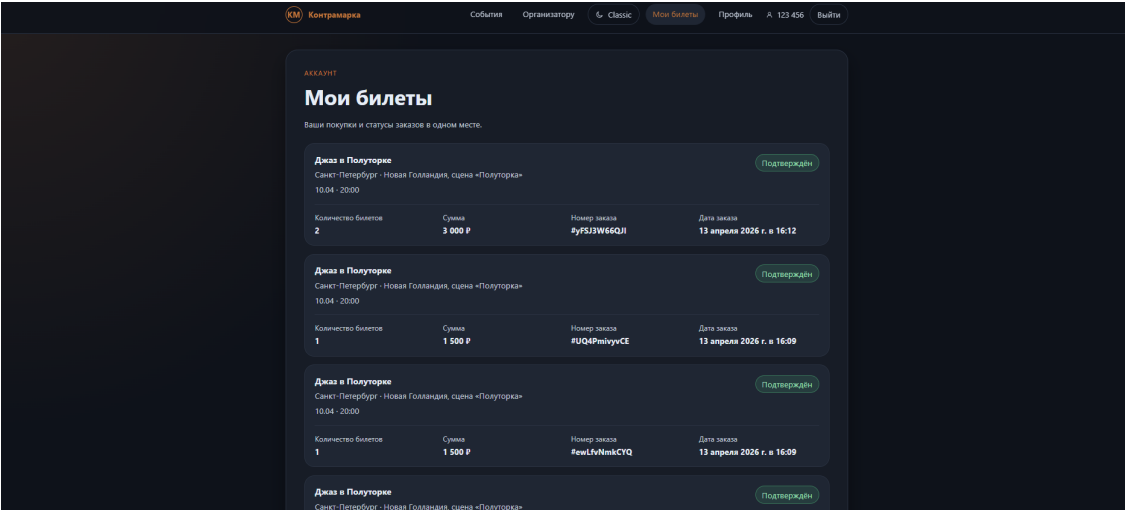
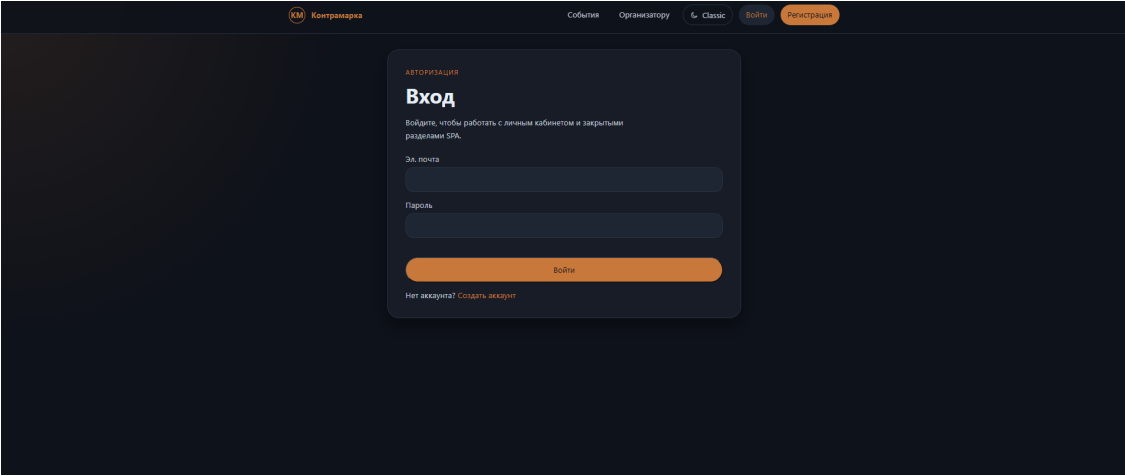
Реализован основной layout приложения с `AppHeader`, `RouterView` и `AppFooter`.

2.2 Настройка маршрутизации

С помощью Vue Router реализованы маршруты:

- главная страница;
- страница события;
- страницы входа и регистрации;
- профиль пользователя;
- страница заказов;
- страница организатора;
- страница 404.





Добавлены meta-поля маршрутов:

- `requiresAuth` — для защищённых страниц;
- `guestOnly` — для страниц авторизации.

Реализован navigation guard (`beforeEach`), обеспечивающий:

- запрет доступа к приватным страницам для неавторизованных пользователей;
- редирект на страницу входа с сохранением целевого маршрута;
- запрет доступа к страницам входа/регистрации для авторизованных пользователей.

2.3 Работа с API

Для взаимодействия с сервером (`json-server`) создан отдельный `Axios` instance. Логика запросов вынесена в отдельные модули:

- `events.js` — работа с мероприятиями;
- `auth.js` — авторизация и регистрация;
- `orders.js` — работа с заказами.

Это позволило избежать дублирования кода и изолировать сетевую логику от компонентов.

2.4 Использование composables

Повторяющаяся логика была вынесена в `composables`:

- `useAuth` — управление авторизацией и пользователем;
- `useEvents` — загрузка списка мероприятий и отдельных событий;
- `useOrders` — работа с заказами и покупкой билетов.

В composables реализованы:

- реактивные состояния (`loading`, `error`, `data`);
- методы загрузки и изменения данных;
- обработка ошибок.

2.5 Миграция пользовательского функционала

Из ЛР2 был перенесён основной пользовательский сценарий:

Каталог мероприятий:

- загрузка списка событий с сервера;
- отображение карточек;
- базовая фильтрация и поиск;

Страница события:

- загрузка события по ID;
- отображение информации;
- обработка состояния отсутствия события;

Авторизация:

- вход и регистрация через json-server;
- сохранение пользователя в localStorage;
- восстановление сессии при перезагрузке страницы;

Покупка билетов:

- создание заказа (`POST /orders`);

- обновление количества билетов (PATCH /events/:id);
- обработка ошибок (нет билетов, событие недоступно);

Заказы:

- загрузка заказов пользователя;
- отображение списка;
- обработка пустого состояния;

2.6 Кабинет организатора

Реализована страница организатора:

- просмотр списка событий;
- поиск и фильтрация;
- создание новых событий;
- редактирование существующих;
- управление публикацией;

Дополнительно реализованы:

- базовая статистика (активные события, продажи);
- локальные настройки через localStorage;

2.7 Компонентный подход

Интерфейс разбит на переиспользуемые компоненты:

- AppHeader, AppFooter;
- EventCard;

- `PurchasePanel`;
- `OrderCard`;
- `EventEditorForm`;

Это позволило:

- сократить дублирование;
- упростить поддержку;
- изолировать логику отдельных частей интерфейса.

2.8 Финальная доработка

На завершающем этапе были выполнены:

- унификация состояний `loading/error/empty`;
- улучшение UX переходов и обновления данных;
- обработка 404-страниц;
- очистка кода от дублирования и временных решений;
- доведение интерфейса до целостного состояния.

3 Выводы

В ходе работы было разработано полноценное SPA-приложение на Vue 3 на основе ранее созданного многостраничного сайта.

Были освоены:

- принципы построения SPA;
- маршрутизация с использованием Vue Router;
- работа с внешним API через Axios;

- компонентный подход;
- выделение переиспользуемой логики с помощью composables.

Миграция приложения позволила:

- повысить переиспользуемость кода;
- упростить навигацию;
- улучшить пользовательский опыт;
- сделать архитектуру приложения более структурированной и масштабируемой.

В результате получено целостное веб-приложение, соответствующее требованиям лабораторной работы.